ASSIGNMENT 2

Optimization Using Python

General instructions

- Use Python with NumPy and Matplotlib where specified. Implement the requested optimizers yourself; do not use an optimization library to perform the iterations.
- Unless a sub-question explicitly permits a library routine, show the formulas and computations used by your implementation.
- Label all axes, include legends where multiple curves or paths are plotted, and use readable plot titles.

Question 1. Exact Line Search for a Quadratic Objective

Use NumPy and Matplotlib to implement and investigate exact line search for the quadratic objective

$$J(w) = \tfrac{1}{2}\, w^T A w + b^T w.$$

(a) Implement the function below. [2 marks]

`exact_line_search_quadratic(A, b, w0, num_steps)`

Inputs:

- A: an n × n symmetric positive-definite matrix
- b: a NumPy array of length n.
- w0: a NumPy array of length n containing the initial point.
- num_steps: a non-negative integer giving the maximum number of iterations.

At iteration t, calculate the negative gradient direction and exact step size:

$$g_t = -(Aw_t + b), \quad \alpha_t = (g_t^T g_t)/(g_t^T A g_t).$$

$$w_{t+1} = w_t + \alpha_t g_t.$$

Stop if $||Aw_t + b||_2 < 10^{-10}$ or after num_steps iterations, whichever occurs first. Avoid dividing by zero if the gradient is numerically zero.

Return: a Python list named iterates containing copies of the initial point and every point generated. Thus iterates[0] is w_0, iterates[t] is w_t, and the list has at most num_steps + 1 entries.

(b) Use the following data.

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 8 \end{bmatrix}, \quad b = [0, 0]^T, \quad w_0 = [2.0, 1.0]^T.$$

$$\text{Therefore, } J(w_1, w_2) = w_1^2 + 4w_2^2.$$

1. Run the function with num_steps = 10. Print w_0 and every iterate returned by the function. [0.5M]
2. Print $J(w_t)$ for every returned iterate w_t. [0.5M]
3. For t = 0, 1, 2, 3, 4, calculate and print $g_t^T g_{t+1}$. Define each g_t using $g_t = -(Aw_t + b)$, and state whether the dot products are approximately zero within floating-point precision. [0.5M]

(c) Plot the optimization path. [1M]

Generate a two-dimensional contour plot of $J(w_1, w_2) = w_1^2 + 4w_2^2$. Overlay the path $w_0 \to w_1 \to \cdots \to w_{10}$ (or all iterates returned if the stopping criterion is met early).

- Mark and label the initial point, the iterates, and the minimizer (0, 0).
- Draw line segments joining consecutive iterates.

- At every intermediate iterate for which both adjacent path segments exist, visually mark the angle between the incoming segment $w_t - w_{t-1}$ and outgoing segment $w_{t+1} - w_t$. Indicate the expected approximately 90° turn; do not describe this as a contour point of tangency.
- Include axis labels, a title, a legend, and a grid.

Question 2. Gradient Descent, Momentum and RMSProp

Consider the loss function

$$J(w_1, w_2) = {w_1}^2 + 10{w_2}^2.$$

(a) Implement the following three optimizers from scratch using NumPy. Each function must return the list of iterates, including the initial point w_0 and the point after each update. [3 marks]

- `vanilla_gd(grad_fn, w0, α, num_iters)`
- `momentum_gd(grad_fn, w0, α, β, num_iters)`
- `rmsprop_gd(grad_fn, w0, α, ρ, ε, num_iters)`

Use grad_fn(w) to obtain the gradient at w. For vanilla gradient descent, update w using $w \leftarrow w - \alpha\ \text{grad_fn}(w)$. For momentum, use velocity $v_0 = 0$ and $v_{t+1} = \beta v_t + \text{grad_fn}(w_t)$, followed by $w_{t+1} = w_t - \alpha v_{t+1}$. For RMSProp, initialize the running squared-gradient average $s_0 = 0$, update $s_{t+1} = \rho s_t + (1 - \rho)(\text{grad_fn}(w_t) \odot \text{grad_fn}(w_t))$, and update $w_{t+1} = w_t - \alpha\ \text{grad_fn}(w_t)/(\sqrt{s_{t+1}} + \varepsilon)$, with operations performed element-wise.

Here, $\odot$ and division are element-wise operations; ε is a small positive constant to avoid division by zero.

(b) Run and plot the optimizers.

1. Initialize all three methods at $w_0 = [3.0, 1.0]^T$ and run 50 iterations with learning rate $\alpha = 0.05$. Use the same initial point and iteration count for all methods. Choose and state the values of β, ρ and ε used for Momentum and RMSProp. [1M]

2. Plot $J(w_t)$ against iteration t for all three methods on a single plot with a logarithmic vertical axis. If a loss reaches zero numerically, handle the log-scale display without taking log(0). [1.5M]

3. Plot the three optimization paths side-by-side over contour lines of $J(w_1, w_2)$. Use the same contour levels and axis limits in all three panels. [1M]
